

iSAQB Module: Formal Methods - Proof Assistants

Active Group GmbH

Created: 2026-10-04 Sun 12:49

Proof assistants

- Proof assistants are software systems that *assist* in constructing formal *proofs*
- Sometimes also called *interactive theorem provers*
 - But: Tools like SMT solvers are sometimes referred to as interactive theorem provers but are generally not considered proof assistants

Assistance in proof

- A human defines formal specifications in the language of the proof assistant
- Generally, proof assistants require a human operator to (partially) construct or at least outline proofs
- The proof assistant checks if the given proof is valid
- The proof assistant helps in constructing proofs
 - descriptive error messages
 - inspection of proof obligations
 - tactics
 - ...

Isabelle

- Isabelle/HOL: Proof assistant developed in Munich and Cambridge

```
section <Finite sequences>

theory Seq
  imports Main
begin

datatype 'a seq = Empty | Seq 'a "'a seq"

fun conc :: "'a seq ⇒ 'a seq ⇒ 'a seq"
where
  "conc Empty ys = ys"
| "conc (Seq x xs) ys = Seq x (conc xs ys)"

fun reverse :: "'a seq ⇒ 'a seq"
where
  "reverse Empty = Empty"
| "reverse (Seq x xs) = conc (reverse xs) (Seq x Empty)"

lemma conc_empty: "conc xs Empty = xs"
  by (induct xs) simp_all

lemma conc_assoc: "conc (conc xs ys) zs = conc xs (conc ys zs)"
  by (induct xs) simp_all

consts
  conc :: "'a seq ⇒ 'a seq ⇒ 'a seq"
Found termination order: "(λp. size (fst p)) <*mlex*> {}"

consts
  reverse :: "'a seq ⇒ 'a seq"
Found termination order: "(λp. size (fst p)) <*mlex*> {}"
```

Seq.thy (\$ISABELLE_HOME/src/HOL/Examples/)

Search: ☐ Ignore case ☐ Regular expressions ☐ HyperSearch ☐ Whole word

Filter:

Seq.thy

- section <Finite sequences>
- theory Seq
- datatype 'a seq = Empty | Seq 'a "'a seq"
- fun conc :: "'a seq ⇒ 'a seq ⇒ 'a seq"
- fun reverse :: "'a seq ⇒ 'a seq"
- lemma conc_empty: "conc xs Empty = xs"
- lemma conc_assoc: "conc (conc xs ys) zs = conc xs (conc ys zs)"
- lemma reverse_conc: "reverse (conc xs ys) = conc (reverse xs) (reverse ys)"
- lemma reverse_reverse: "reverse (reverse xs) = xs"

☐ Proof state ☒ Auto hovering ☒ Auto update Search: 100%

consts

conc :: "'a seq ⇒ 'a seq ⇒ 'a seq"

Found termination order: "(λp. size (fst p)) <*mlex*> {}"

Output

14,6 (214/797) jelle UTF-8-Isabelle Wrap: none Multi sel: false Rect sel: false Overwrite: false LF global JVM: 944/944MiB ML: 13/498MiB 9:32 PM

Lean

- **Lean**: Proof assistant and functional programming language based on dependent types

```
example (x y : Int) :  
  27 ≤ 11*x + 13*y → 11*x + 13*y ≤ 45  
  → -10 ≤ 7*x - 9*y → 7*x - 9*y > 4 := by  
  grind
```

Agda

- **Agda**: Proof assistant and functional programming language based on dependent types

```
step-y-correct s@(st dx@(n+1 dx-1) dy x y (ℤ.negsuc n)) y-corr =
  begin
  to-rat (state-y (step (st dx dy x y (ℤ.negsuc n)))) -q ½
    ≡{ } -- expansion of definitions
  to-rat y -q ½
    ≤{ y-corr }
  m *q to-rat x
    ≤{ ≤-stepr 0≤m } -- make rhs larger
  (m *q to-rat x) +q m
    ≡{ cong (λ p -> (m *q to-rat x) +q p) (sym (*-identityr m)) } -- * right identity
  (m *q to-rat x) +q (m *q 1ℚ)
    ≡{ sym (*-distribl -+ m (to-rat x) 1ℚ) } -- distributivity
  (m *q ((to-rat x) +q 1ℚ))
    ≡{ cong (λ p -> m *q p) (lem-3 x) }
  m *q to-rat (suc x)
    ≡{ }
  state-m (step (st dx dy x y (ℤ.negsuc n))) *q
  to-rat (state-x (step (st dx dy x y (ℤ.negsuc n))))
  ■
where
  open ℚp.≤-Reasoning
  m = state-m s
  0≤m = 0≤state-m s
```

Rocq

- **Rocq**: Proof assistant and functional programming language based on dependent types
- formerly: Coq (after Thierry Coquand)

```
Fixpoint fac n :=  
  match n with  
  | 0 => 1  
  | S n' => n * fac n'  
end.
```

Compute fac 5.

```
Lemma fac_succ n : fac (S n) = S n * fac n.  
Proof. reflexivity. Qed.
```

```
Lemma fac_pos n : fac n > 0.  
Proof.  
  induction n.  
  - simpl. lia.  
  - rewrite fac_succ. lia.  
Qed.
```

Others

- ACL2
- F*
- Idris
- NuPRL
- ...